Different Types of Specs

Our goal as developers is to write specs that maximize the values we've listed here—guiding the design, building confidence, and so on—while minimizing the time lost to writing, running, and fixing them.

Every spec has a job to do. These jobs fall into different categories: catching regressions in an application, guiding the design of a single class or method, and so on.

The software development community continually argues about how many of these categories there are, and their exact definitions. While these endless arguments and subcategories are fun to ponder, we recommend focusing on just a few different, well-defined types of specs. That way, you'll end up *intentionally* picking what to write at any given moment, based on the benefit you're pursuing.

For this book, we are going to consider three types of specs described by Steve Freeman and Nat Pryce in *Growing Object-Oriented Software, Guided by Tests [FP09]*:

- *Acceptance:* Does the whole system work?
- *Unit:* Do our objects do the right thing, are they convenient to work with?
- *Integration:* Does our code work against code we can't change?

Let's look at each of these types of specs one by one.

Acceptance Specs

Acceptance specs describe a feature in an end-to-end, black-box style that exercises the entire system. These specs are hard to write, comparatively brittle, and slow. But they also provide a great deal of confidence that the parts of the system are working together as a whole. They're also supremely useful for large-scale refactoring.

Unit Specs

At the other end of the spectrum, unit specs focus on individual units of code—often as small as a single object or method. They check the behavior of a piece of code *relative to the environment you construct for it.*

Well-written unit specs tend to run extremely quickly (often in a few milliseconds or less!), and thus tend to cost less than other kinds of specs. Their isolated, focused nature provides immediately useful design feedback. Their independent nature makes them less likely to interfere with one another in a large suite.